

# PREDICTION OF FOOTBALL MATCH RESULTS USING MACHINE LEARNING MODELS

A DISSERTATION SUBMITTED TO MANCHESTER METROPOLITAN UNIVERSITY
FOR THE DEGREE OF MASTER OF SCIENCE

DEPARTMENT OF COMPUTING AND MATHEMATICS
IN THE FACULTY OF SCIENCE AND ENGINEERING

By Chathura Gamage (ST ID: 22542695)

Supervised by: Dr. Xia Cui

# Overview

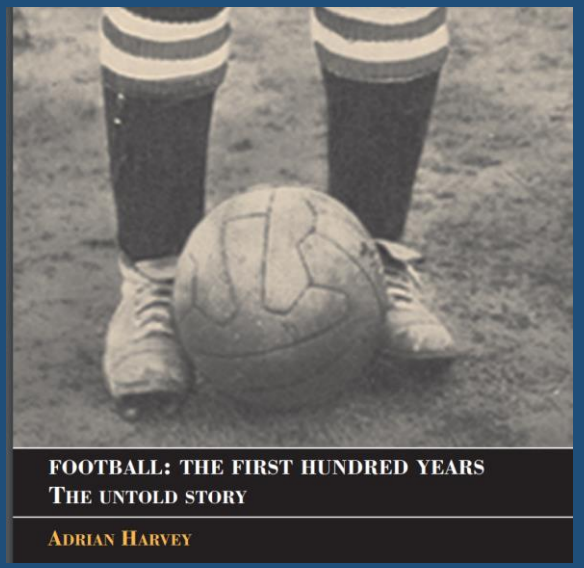



1. Introduction
2. The Literature Review
3. Understanding the
4. Data Preprocessing
6. Methodology & Model Design
7. Implementation & Results
8. Model Evaluation
9. Iterative Improvements
9. Conclusion & Future Work

# Introduction



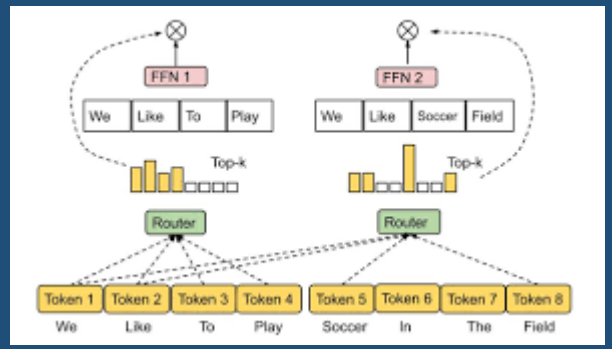
- Sports activities are inherently relieving internal aggressions in a peaceful manner.
- The game of football has been a world-renowned sport since 1930 and has drawn huge attention.
- The predictions are needed in coping with the future and are always challenging with chance events.
- The situation demanded a robust prediction means.
- AI solutions of machine learning algorithms have become versatile



# Potential Problem

- Renown sport-football
- So, diverse stakeholders
- AI-based machine learning for more accurate predictions.
- A robust application has been a tough ask.
- Endless process with competitiveness &...
- Hence,……

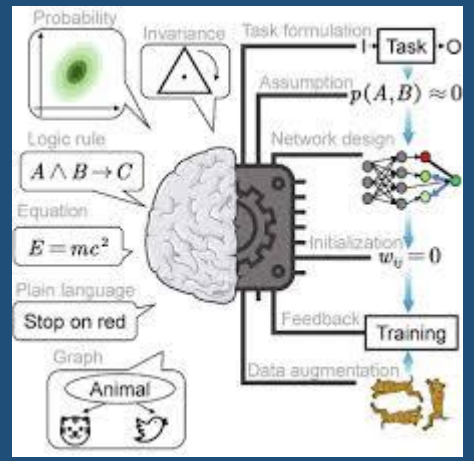


# Aim & Objectives

**Aim**

- Predicting results of football matches by ML models to make decisions on future games.




**Objectives**

a. Collect an appropriate dataset, Understand the tasks and features.

b.. Process the data & and model application.

c. Selecting potentially contributed features.

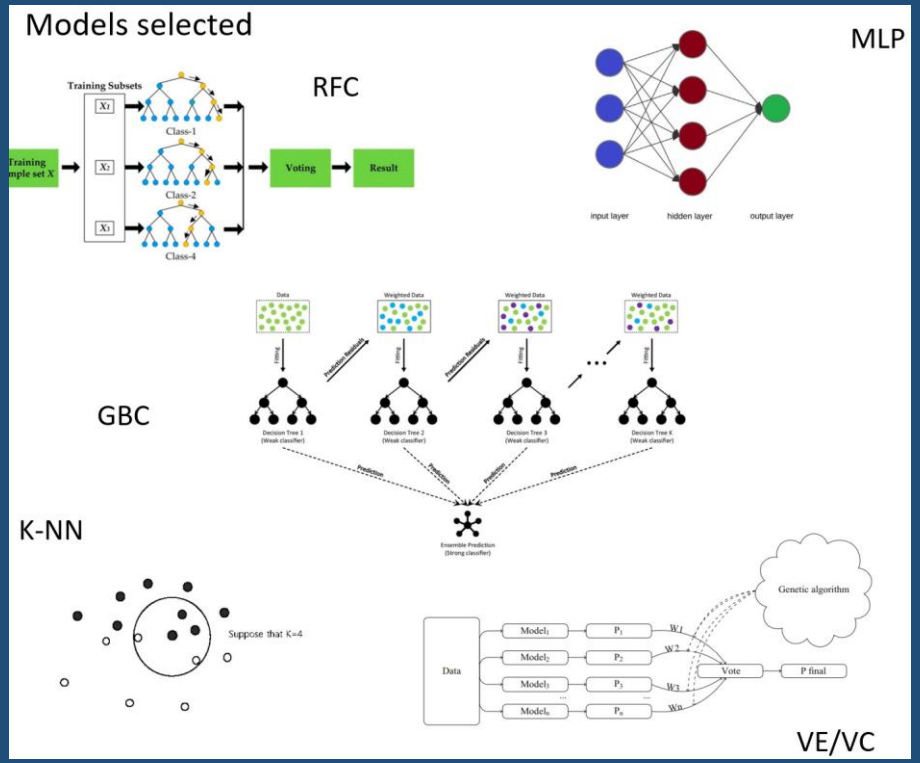
d. Train ML models for the prediction of the football match results,.

e. Validate the prediction with the actual.

f. Record and compile the report.

# Literature Review

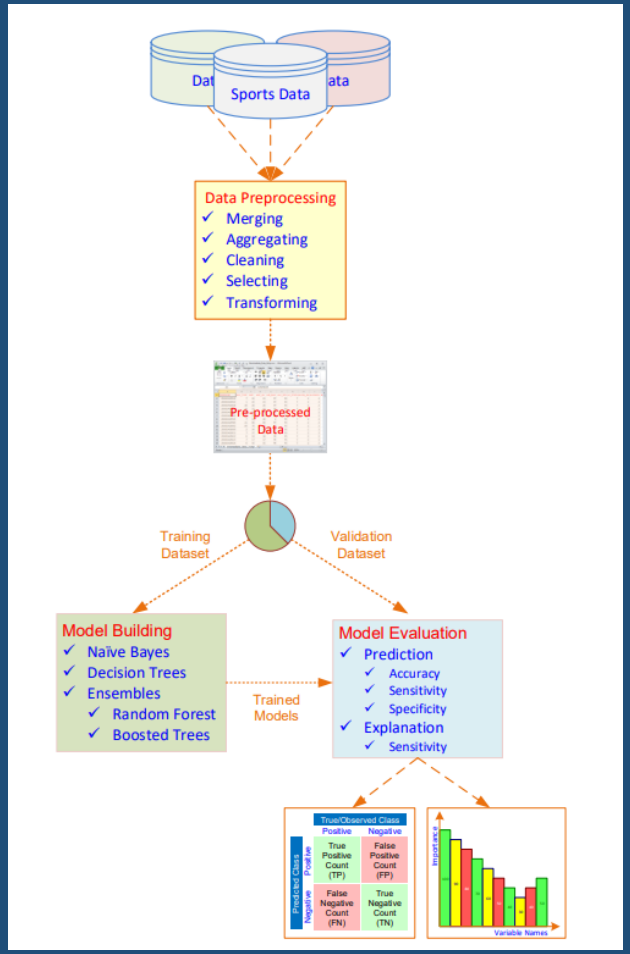
- ML Algorithms in use
  - SVM
  - LR
  - GNB
  - RF
  - kNN
  - GB
  - ANN
    - MLP
    - CNN
  - VE(VC)



# Literature Review cont…

**Other Empirical evidence**

- Preprocessing
  - Feature engineering
  - Feature Selection strategies
  - ELO rating
  - Clustering
  - Data splitting
- Methodologies adapted
- Evaluation Matrix



# Understand Data

- Data Exploration
  - Summary statistics
  - Performance analysis
    - Win-Loss analysis
  - Correlation analysis
  - Analysis of betting odds
  - Feature importance
  - Visualization



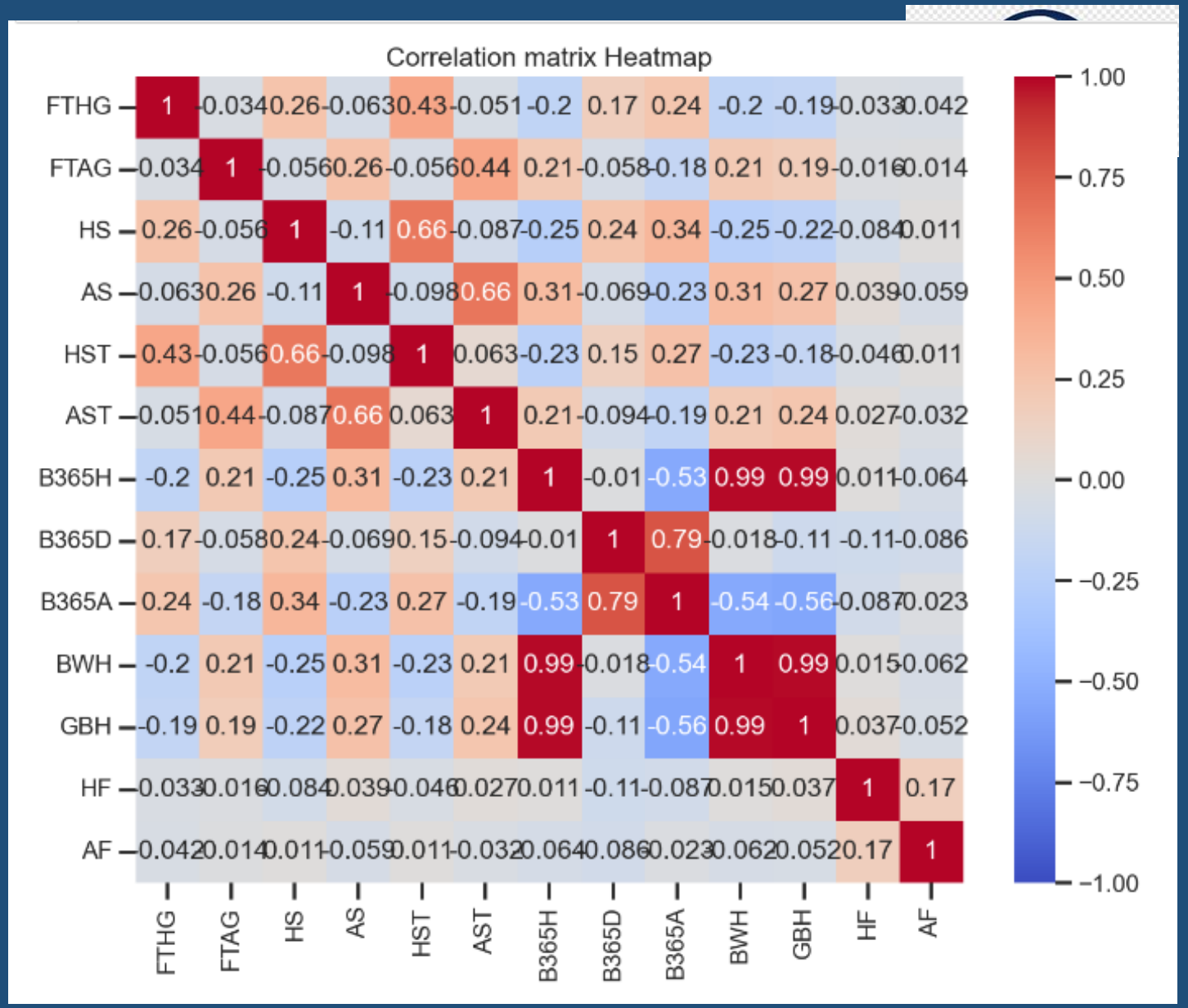
```
1  fbdata.count()
```

```
Date          45952
HomeTeam      45952
AwayTeam      45952
FTHG          45952
FTAG          45952
              ...
PCAHA          9841
MaxCAHH        9856
MaxCAHA        9856
AvgCAHH        9856
AvgCAHA        9856
Length: 138, dtype: int64
```

```
1  # Identify the data types
2  fbdata.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 45952 entries, E0 to EC
Columns: 138 entries, Date to AvgCAHA
dtypes: float64(130), object(8)
memory usage: 48.7+ MB
```

# Understand Data cont…




Correlation Matrix

# Understand Data cont…



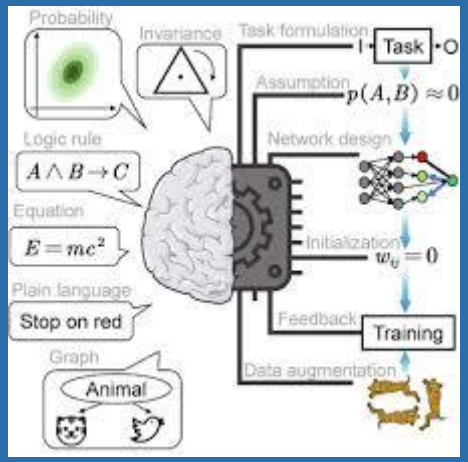
Win Analysis

## 2.4. Win loss Analysis

```
1 outcome_frequency = fbdata['FTR'].value_counts()
2 print(outcome_frequency)
```

```
H    19936
A    13892
D    12124
Name: FTR, dtype: int64
```

# Data Preprocessing



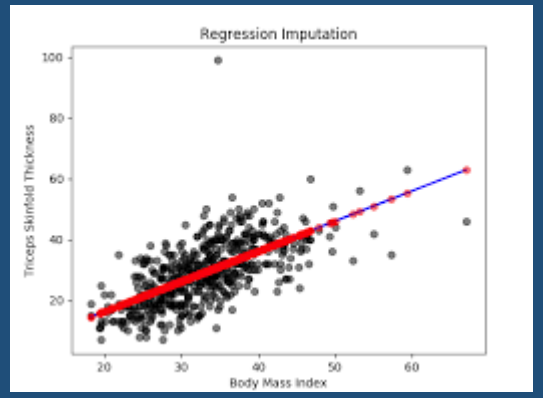

- Missing Value Imputation
- Feature Engineering
- ELO Rating
- Clustering
- Feature Selection
- Scaling and encoding

# Data Preprocessing cont…

## Missing Value Imputation



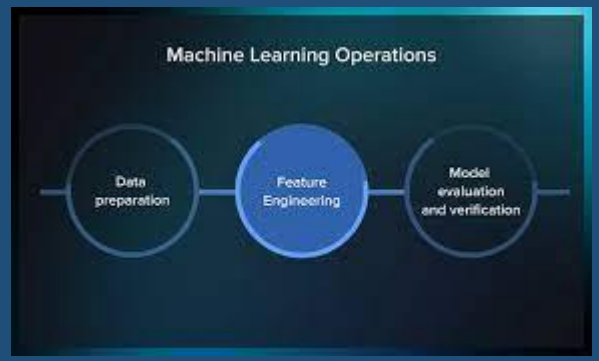
- Numerical Features
  Replaced with mean, bfill methods
- Categorical Features
  - Replaced with mode



# Data Preprocessing cont…

## Feature Engineering

- Derive new features




Ex

- Averaging Betting odds
- Goal gap
- Home win streak
- Average Goals conceded
- Points Away Team
- Points Home Team
- Age, Weighted Age
- Form Indicator
- Dropping unrelated columns

# Data Preprocessing cont…



- Ex. Team Form Indicator
  - Got the last 5 match performances.
  - Got the point gap



| Age | HomeTeam | AwayTeam | FormIndicator |
|---|---|---|---|
| 0 | Torquay | Wrexham | 6 |
| 0 | Derby | Morecambe | -7 |
| 0 | Exeter | Charlton | -7 |
| 0 | Fleetwood Town | Burton | -7 |
| 0 | Forest Green | Peterboro | -7 |
| ... | ... | ... | ... |
| 10 | Crawley Town | Port Vale | -7 |
| 10 | Crewe | Stevenage | -7 |
| 10 | Gillingham | Notts County | -7 |
| 10 | Oldham | Carlisle | -7 |
| 10 | Arsenal | Aston Villa | 7 |

398 rows x 4 columns]

# Data Preprocessing cont…




**Betting Odds**

**Win/ Draw/  ods**

**Max odds**

**Odds over 2.5 goals**

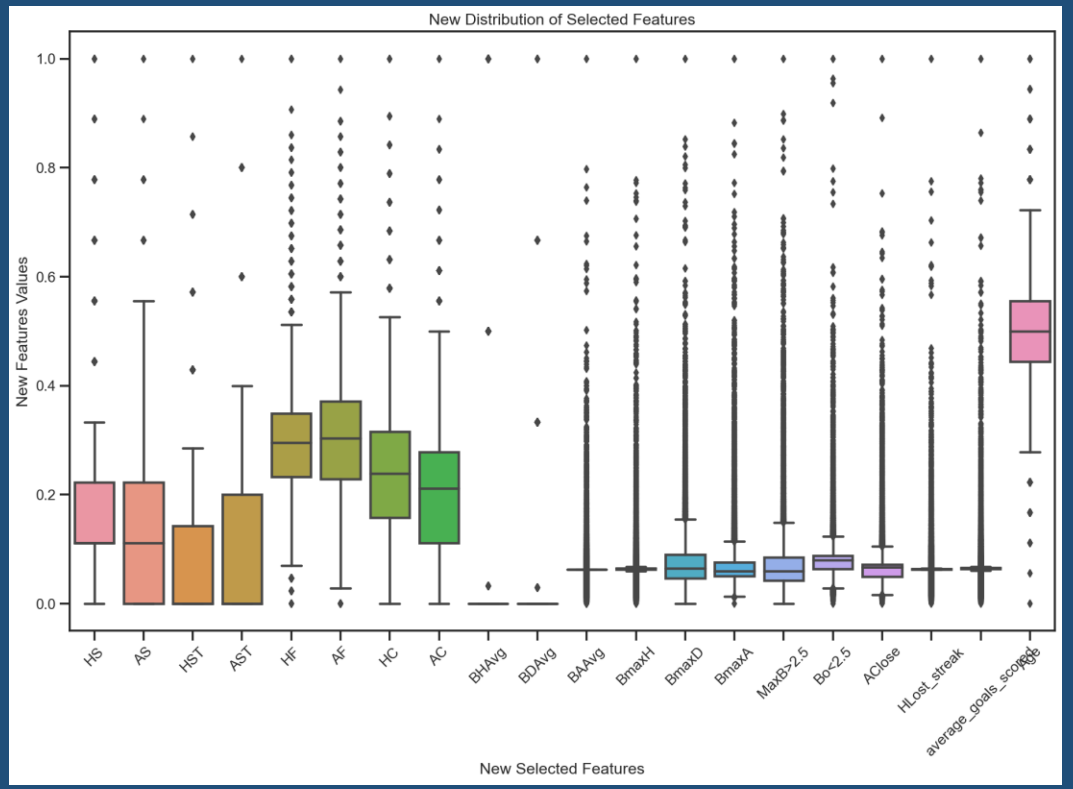
- Congestion of features.
- Grouped
- Averaged out

HS', 'AS', 'HST', 'AST', 'HC', 'HY', 'AvgCH', 'AvgCA', 'BHAvg', 'BDAvg', 'BAAvg', 'BmaxH', 'BmaxD', 'BmaxA', 'HClose', 'AClose', 'goal_gap', 'Hwin_streak', 'H

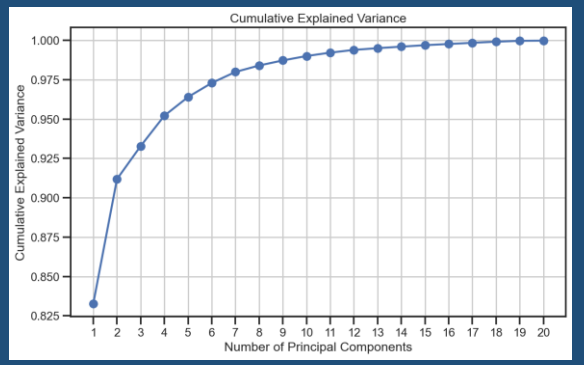
# Data Preprocessing cont…-

## Feature Selection

- **Assess correlation with correlation matrix**
- **SelectKBest (20)**:




PCA Threshold: 18 features



# Methodology and Model Design

Preprocessing

**Model Selection**:

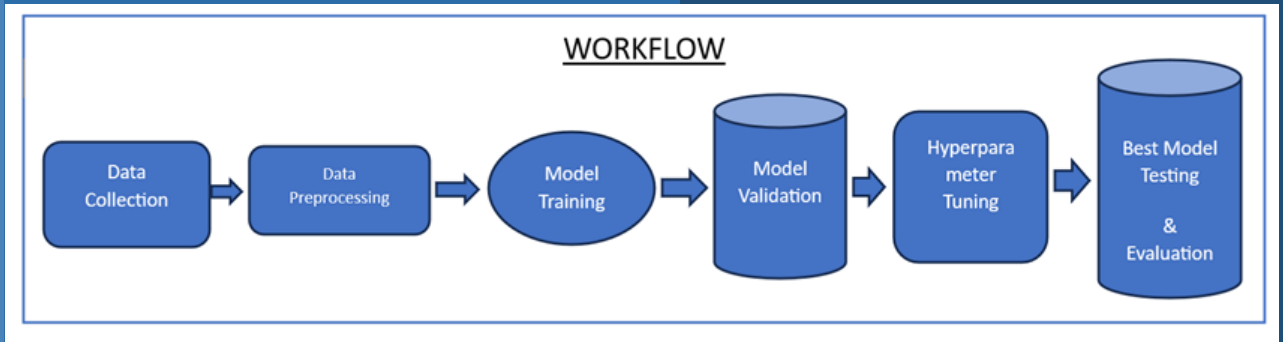
RFC

k_NN

MLP

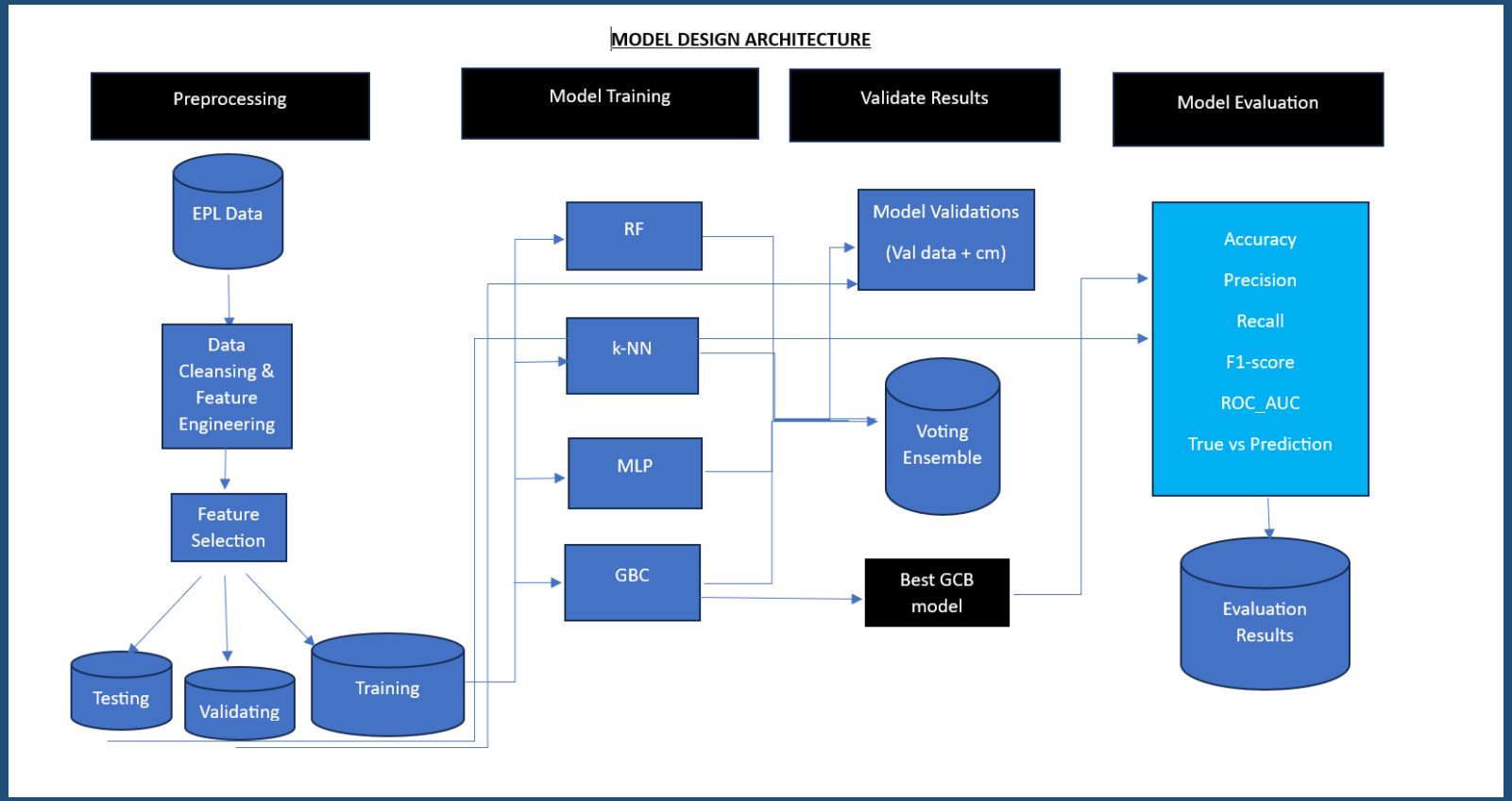
GBC

VC




- Models trained on training data
- Validated on validation data
- Hyperparameter tuned with Grid Search CV
- Tested the best model VE,  on testing data
- Evaluation of robust model

# Methodology and Model Design cont…



# Implementation and Results cont…

RFC Model

Initial base model

Accuracy: 0.55
RFC Classification Report for validation data:

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| A | 0.55 | 0.60 | 0.57 | 1585 |
| D | 0.38 | 0.12 | 0.18 | 1327 |
| H | 0.58 | 0.79 | 0.67 | 2193 |
| accuracy | | | 0.55 | 5105 |
| macro avg | 0.50 | 0.50 | 0.47 | 5105 |
| weighted avg | 0.52 | 0.55 | 0.51 | 5105 |

Hyperparameter tuned

```
# Hyperparameter tuning
# Accuracy=0.55; max_features = 'sqrt', class_weight= None, min_samples_split= 5, min_samples_leaf= 4 ,max_depth = 40, n_est
# Accuracy=0.55; max_features = 'sqrt', class_weight= None, min_samples_split= 5, min_samples_leaf= 4 ,max_depth = 10, n_est
# Accuracy=0.54; max_features = 'sqrt', class_weight= None, min_samples_split= 5, min_samples_leaf= 4 ,max_depth = 40, n_est
# Accuracy=0.55; max_features = 'sqrt', class_weight= None, min_samples_split= 5, min_samples_leaf= 4 ,max_depth = 40, n_est
# Accuracy=0.54; max_features = 'sqrt', class_weight= None, min_samples_split= 10, min_samples_leaf= 4 ,max_depth = 40, n_es
# Accuracy=0.55; class_weight= None, min_samples_split= 10, min_samples_leaf= 4 ,max_depth = 40, n_estimators=50.
# Accuracy=0.55; max_features = 'sqrt', min_samples_split= 10, min_samples_leaf= 4 ,max_depth = 40, n_estimators=50.

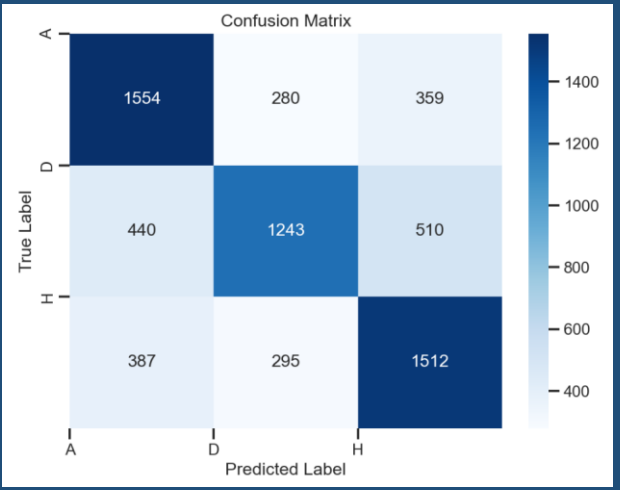
# For tuning hyper parameters, define the grid
rf_param_grid = {
    'n_estimators': [50,150,200],   # Number of trees in the forest
    'max_depth' : [20, 40],    # The max depth of trees
    #'min_samples_split': [5,10],  # Max no of samples required to split internal node
    #'min_samples_leaf' : [4, 8],   # Max no of samples required to be at a leaf node
    #'max_features': ['sqrt', None], # No of features in looking fo the best split
    'class_weight' : [None, 'balanced']       # weight to address class imbalance
}
```

Verified

Resampled with SMOTE

RFC Classification Report for Validation data:

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| A | 0.65 | 0.71 | 0.68 | 2193 |
| D | 0.68 | 0.57 | 0.62 | 2193 |
| H | 0.64 | 0.69 | 0.66 | 2194 |
| accuracy | | | 0.65 | 6580 |
| macro avg | 0.66 | 0.65 | 0.65 | 6580 |
| weighted avg | 0.66 | 0.65 | 0.65 | 6580 |



# Implementation and Results cont…

K_NN Model

```
# hyperparameter tuning
# Accuracy: 0.54: n_neighbors= 60, weights= 'distance', p=2
# Accuracy: 0.58: n_neighbors= 40, weights= 'distance', p=2
# Accuracy: 0.58: n_neighbors= 20, weights= 'distance', p=2
# Accuracy: 0.53: n_neighbors= 60, weights= 'distance', p=2
```

Tune

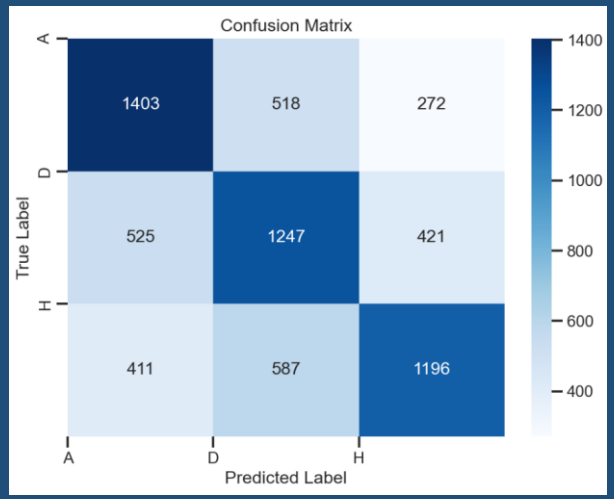


knn Best model Classification Report:

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| A | 0.62 | 0.66 | 0.64 | 2193 |
| D | 0.56 | 0.62 | 0.59 | 2193 |
| H | 0.64 | 0.53 | 0.58 | 2194 |
| accuracy | | | 0.60 | 6580 |
| macro avg | 0.61 | 0.60 | 0.60 | 6580 |
| weighted avg | 0.61 | 0.60 | 0.60 | 6580 |

Best Model

Verified



# Implementation and Results cont…

## MLP Model

Accuracy: 0.61
MLP Base Classification Report:

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| A | 0.62 | 0.65 | 0.64 | 2193 |
| D | 0.59 | 0.61 | 0.60 | 2193 |
| H | 0.62 | 0.57 | 0.59 | 2194 |
| accuracy | | | 0.61 | 6580 |
| macro avg | 0.61 | 0.61 | 0.61 | 6580 |
| weighted avg | 0.61 | 0.61 | 0.61 | 6580 |

Base Model



```
#Hyperparameter tuning
hidden_layer_sizes=(100,50), max_iter=300, Accuracy= 0.48
learning_rate= 'invscaling', alpha=0.0001, solver= 'adam', Accuracy= 0.48.4
activation ='tanh'. accuracy= 49.2
hidden_layer_sizes=(100,50), max_iter= 100, activation ='tanh'. Accuracy= .47
hidden_layer_sizes=(200,300), max_iter= 700, activation ='tanh'. Accuracy= .4884
hidden_layer_sizes=(100,50), max_iter= 500, activation ='tanh'. Accuracy= .4723
hidden_layer_sizes=(200,100), max_iter= 500, activation ='tanh'. Accuracy= .4846
hidden_layer_sizes=(200,300), max_iter= 500, activation ='tanh'. Accuracy= .4884
hidden_layer_sizes=(400,300), max_iter= 500, activation ='tanh'. Accuracy= .4688
hidden_layer_sizes=(300,300), max_iter= 500, activation ='tanh'. Accuracy= .4879
hidden_layer_sizes=(300,200), max_iter= 500, activation ='tanh'. Accuracy= .4738
hidden_layer_sizes=(activation ='tanh'). Accuracy= .4804
batch_size = 64, hidden_layer_sizes=(200,300), max_iter= 500, activation ='tanh'.Accuracy = .4809
batch_size = 32, hidden_layer_sizes=(200,300), max_iter= 500, activation ='tanh'.Accuracy = .4854
learning_rate= 'adaptive', hidden_layer_sizes=(200,300), max_iter= 500, activation ='tanh'.Accuracy = .4884
```

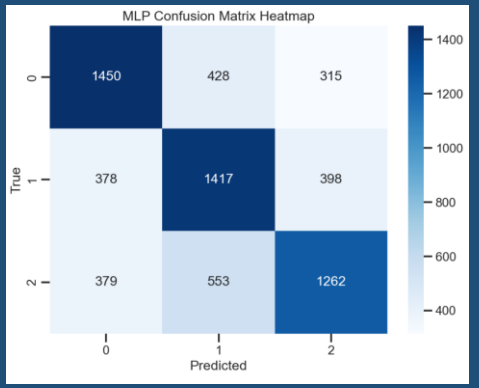
Tuned

Accuracy: 0.627508
MLP Best Classification Report:

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| A | 0.66 | 0.66 | 0.66 | 2193 |
| D | 0.59 | 0.65 | 0.62 | 2193 |
| H | 0.64 | 0.58 | 0.61 | 2194 |
| accuracy | | | 0.63 | 6580 |
| macro avg | 0.63 | 0.63 | 0.63 | 6580 |
| weighted avg | 0.63 | 0.63 | 0.63 | 6580 |

Best Model

Verified



# Implementation and Results cont…

GBC Model

```
Accuracy: 0.58
GBC Base Classification Report:
              precision    recall  f1-score   support

           A       0.57      0.61      0.59      2193
           D       0.53      0.46      0.49      2193
           H       0.63      0.67      0.65      2194

    accuracy                           0.58      6580
   macro avg       0.58      0.58      0.58      6580
weighted avg       0.58      0.58      0.58      6580
```

Base Model



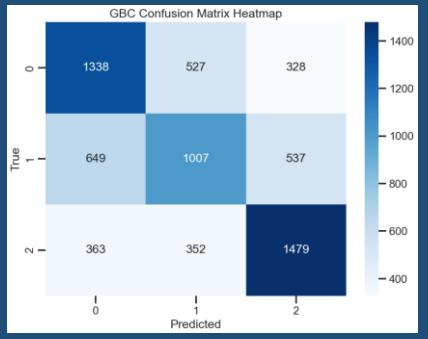
```
Accuracy:  0.581155
 Best GBC Classification Report for:
              precision    recall  f1-score   support

           A       0.57      0.61      0.59      2193
           D       0.53      0.46      0.49      2193
           H       0.63      0.67      0.65      2194

    accuracy                           0.58      6580
   macro avg       0.58      0.58      0.58      6580
weighted avg       0.58      0.58      0.58      6580
```

Best Model

Verified



# Implementation and Results cont…

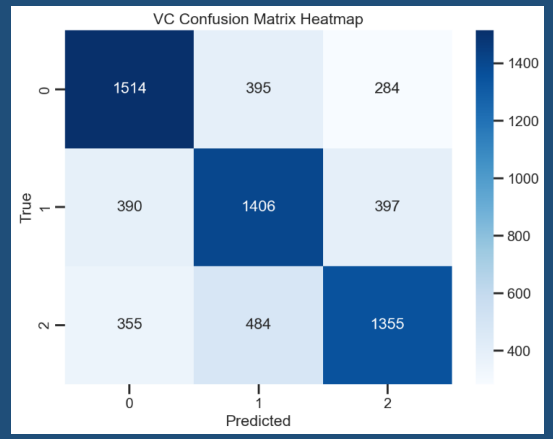
VC Best Model

Accuracy: 0.649696
Voting Classification Report for:

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| A | 0.67 | 0.69 | 0.68 | 2193 |
| D | 0.62 | 0.64 | 0.63 | 2193 |
| H | 0.67 | 0.62 | 0.64 | 2194 |
| accuracy | | | 0.65 | 6580 |
| macro avg | 0.65 | 0.65 | 0.65 | 6580 |
| weighted avg | 0.65 | 0.65 | 0.65 | 6580 |

Best Model

Verified



# Model Evaluation

## Matrix followed



- **Accuracy**= (T P + T N)/ (T P + T N + F P + F N)
  - Ratio of correct predictions to the total number of predictions
- **Precision**= TP/ (TP + F P)
  - how accurately the model's optimistic predictions come true
- **Recall(Sensitivity)** = TP/ (TP + F N)
  - proportion of positive samples correctly identified
- **F1-score**= 2 · precision · recall/ (precision + recall)
  - F1 score is the combination of the measurement of precision and the Recall
- **ROC-AUC**= TPR (Sensitivity against FPR
- **True vs. Prediction**= Actual vs. prediction that reveals the model performance and accuracy, How well the model discriminates between classes
- **SHAP**

# Model Evaluation cont…

```
f{name} ~ testing Accuracy: {accuracy_test: .2f}
Testing Classification Report:
              precision    recall  f1-score   support

           A       0.67      0.70      0.69      1080
           D       0.69      0.59      0.64      1081
           H       0.64      0.70      0.67      1080

    accuracy                           0.66      3241
   macro avg       0.67      0.66      0.66      3241
weighted avg       0.67      0.66      0.66      3241

f{name} ~ testing Accuracy: {accuracy_test: .2f}
Testing Classification Report:
              precision    recall  f1-score   support

           A       0.61      0.64      0.62      1080
           D       0.55      0.58      0.56      1081
           H       0.62      0.55      0.59      1080

    accuracy                           0.59      3241
   macro avg       0.59      0.59      0.59      3241
weighted avg       0.59      0.59      0.59      3241

f{name} ~ testing Accuracy: {accuracy_test: .2f}
Testing Classification Report:
              precision    recall  f1-score   support

           A       0.66      0.65      0.66      1080
           D       0.59      0.66      0.62      1081
           H       0.63      0.57      0.60      1080

    accuracy                           0.63      3241
   macro avg       0.63      0.63      0.63      3241
weighted avg       0.63      0.63      0.63      3241

f{name} ~ testing Accuracy: {accuracy_test: .2f}
Testing Classification Report:
              precision    recall  f1-score   support

           A       0.59      0.64      0.62      1080
           D       0.56      0.46      0.51      1081
           H       0.64      0.70      0.67      1080

    accuracy                           0.60      3241
   macro avg       0.60      0.60      0.60      3241
weighted avg       0.60      0.60      0.60      3241

f{name} ~ testing Accuracy: {accuracy_test: .2f}
Testing Classification Report:
              precision    recall  f1-score   support

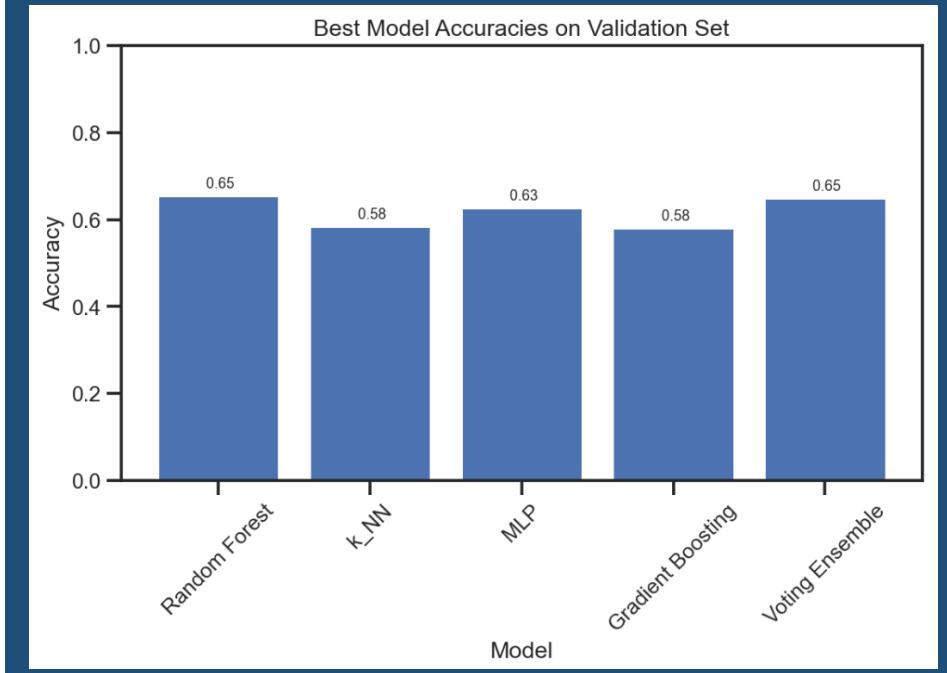
           A       0.67      0.69      0.68      1080
           D       0.63      0.64      0.63      1081
           H       0.65      0.62      0.64      1080

    accuracy                           0.65      3241
   macro avg       0.65      0.65      0.65      3241
weighted avg       0.65      0.65      0.65      3241
```

## Let's test the Best Models by Testing Data



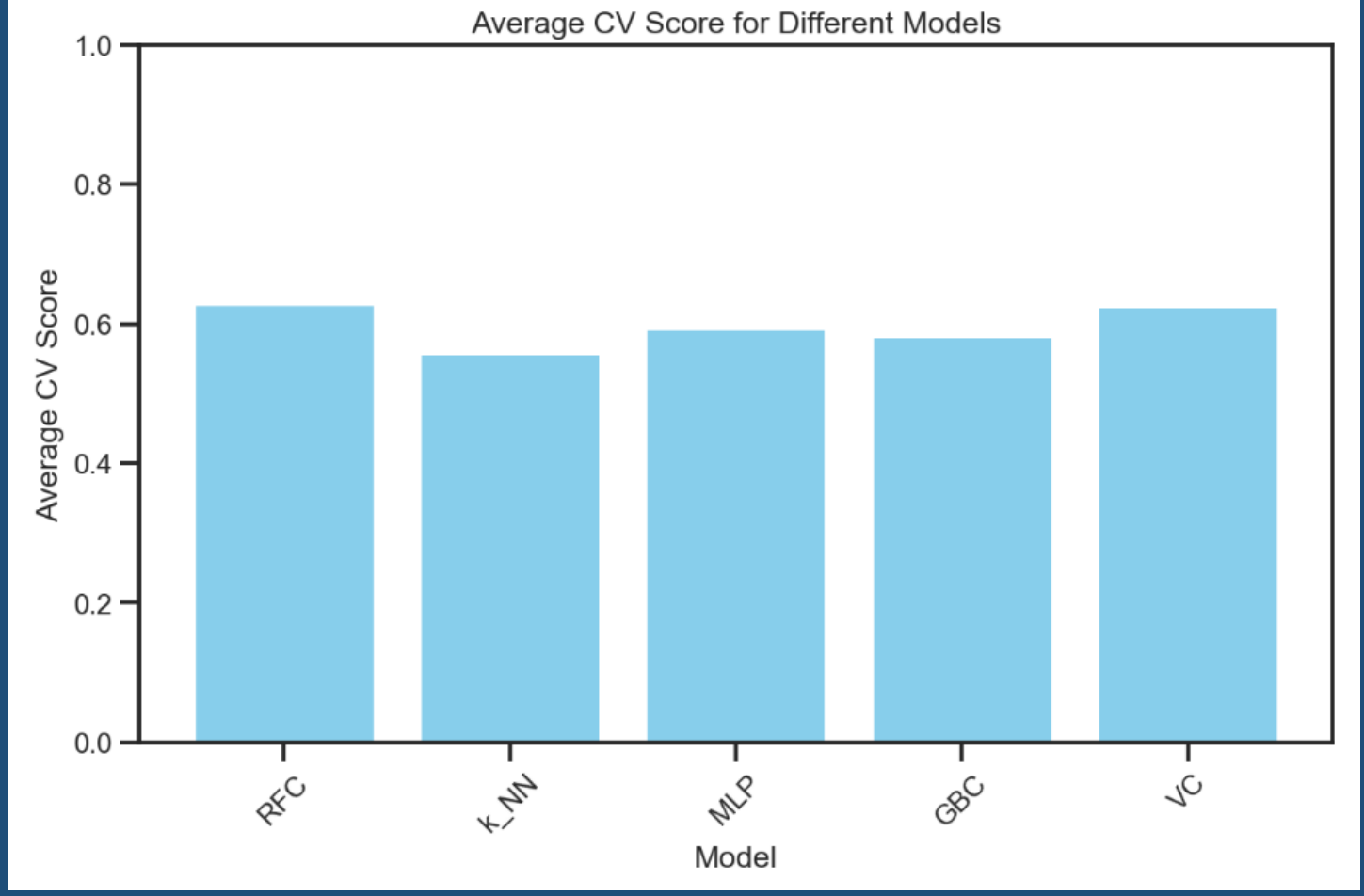
## Compares Tested Best Models



# Model Evaluation cont…

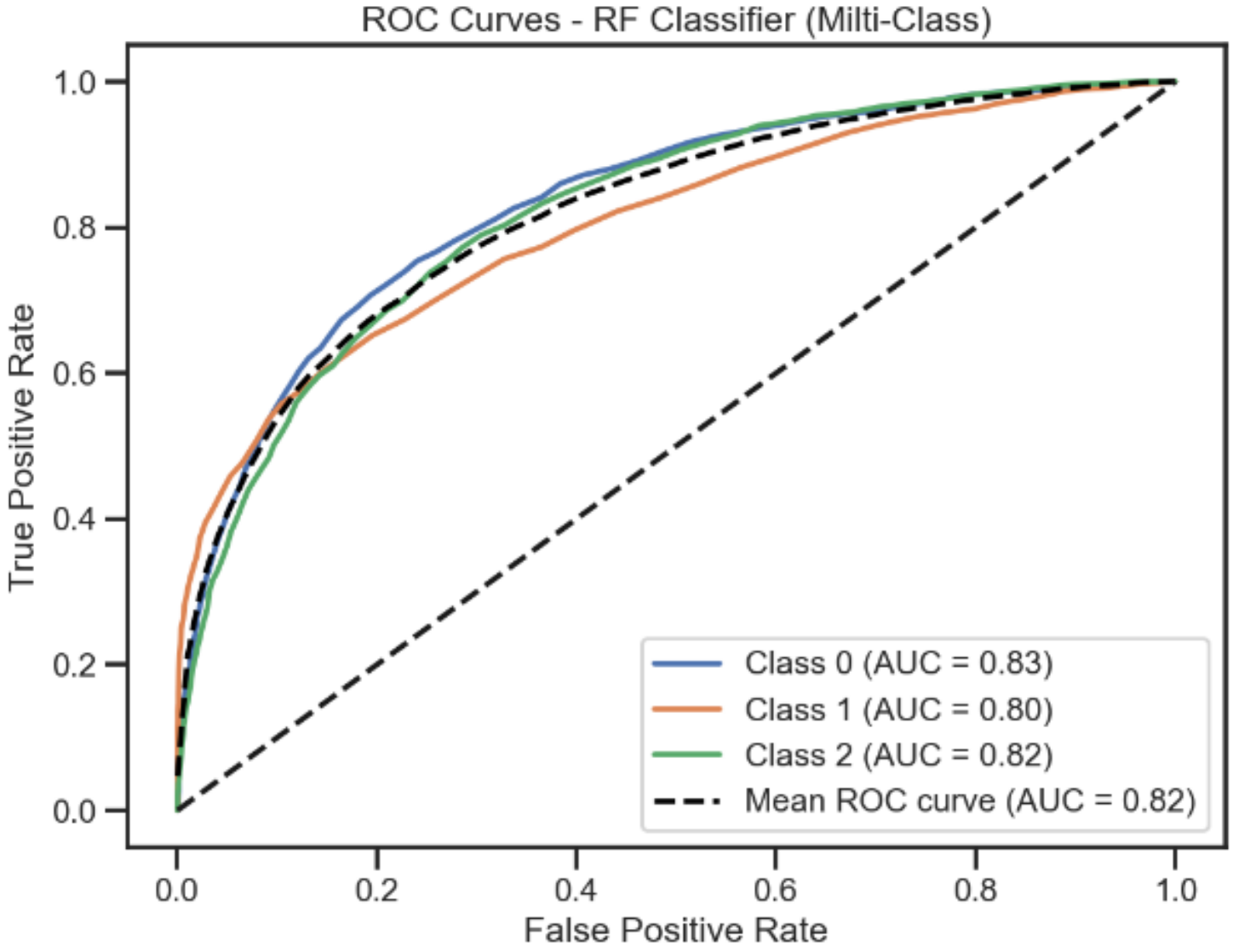
Overall Performance of the models with CV Scores

```
RFC - Average CV Score: 0.6285
k_NN - Average CV Score: 0.5585
MLP - Average CV Score: 0.5939
GBC - Average CV Score: 0.5822
VC - Average CV Score: 0.6254
```



# Model Evaluation cont…

ROC-AUC
0.82




Class Mapping = {'A': 0, 'D': 1, 'H': 2}

# ROC-AUC



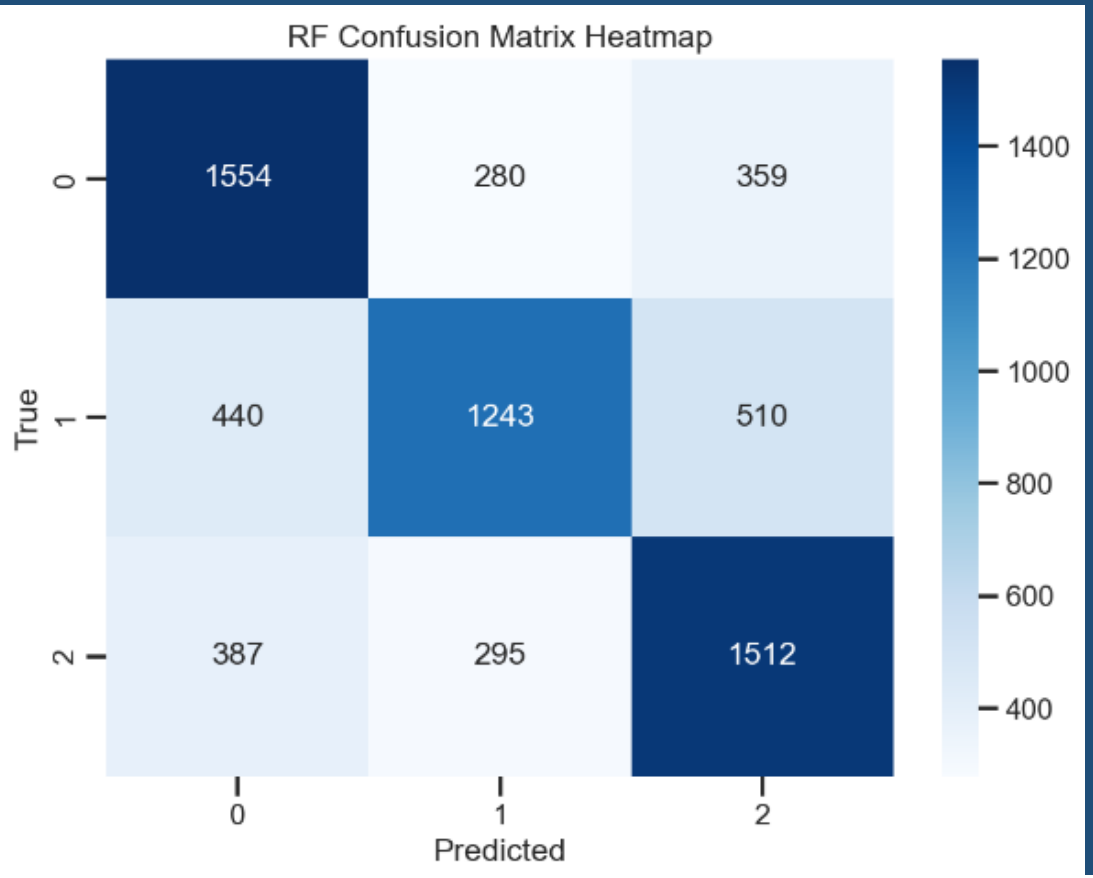
**ROC-AUC**

- Binary classification.
- Plot TPR against FPR for different threshold values.
- TPR (sensitivity or recall).
- Shows how to model sensitivity & and specificity change.

- The diagonal line represents the performance of the Random Classifier.
- AUC represents the overall performance where 1 is the optimum. 085 indicates discriminatory power.
- The intention is to minimize FP and maximize TP.

# Model Evaluation cont…

True vs Predicted analysis of the best model VC

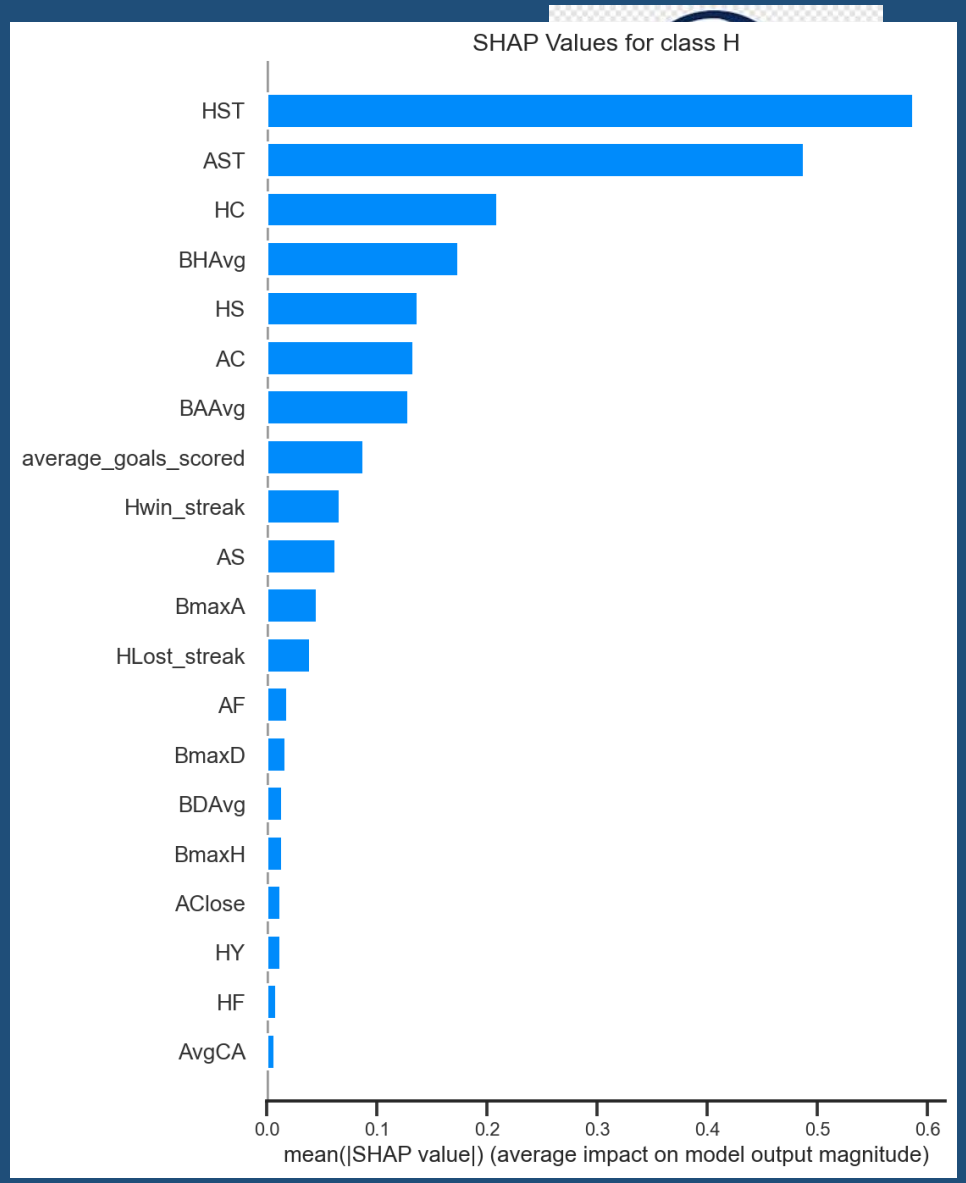



```
Confusion Matrix:
<function confusion_matrix at 0x000002C6BAEB7550>
Accuracy: 0.6548632218844985
Precision: 0.6548632218844985
Recall: 0.6548632218844985
F1 Score: 0.6548632218844985
```

# Model Evaluation cont…

(SHAP Analysis)

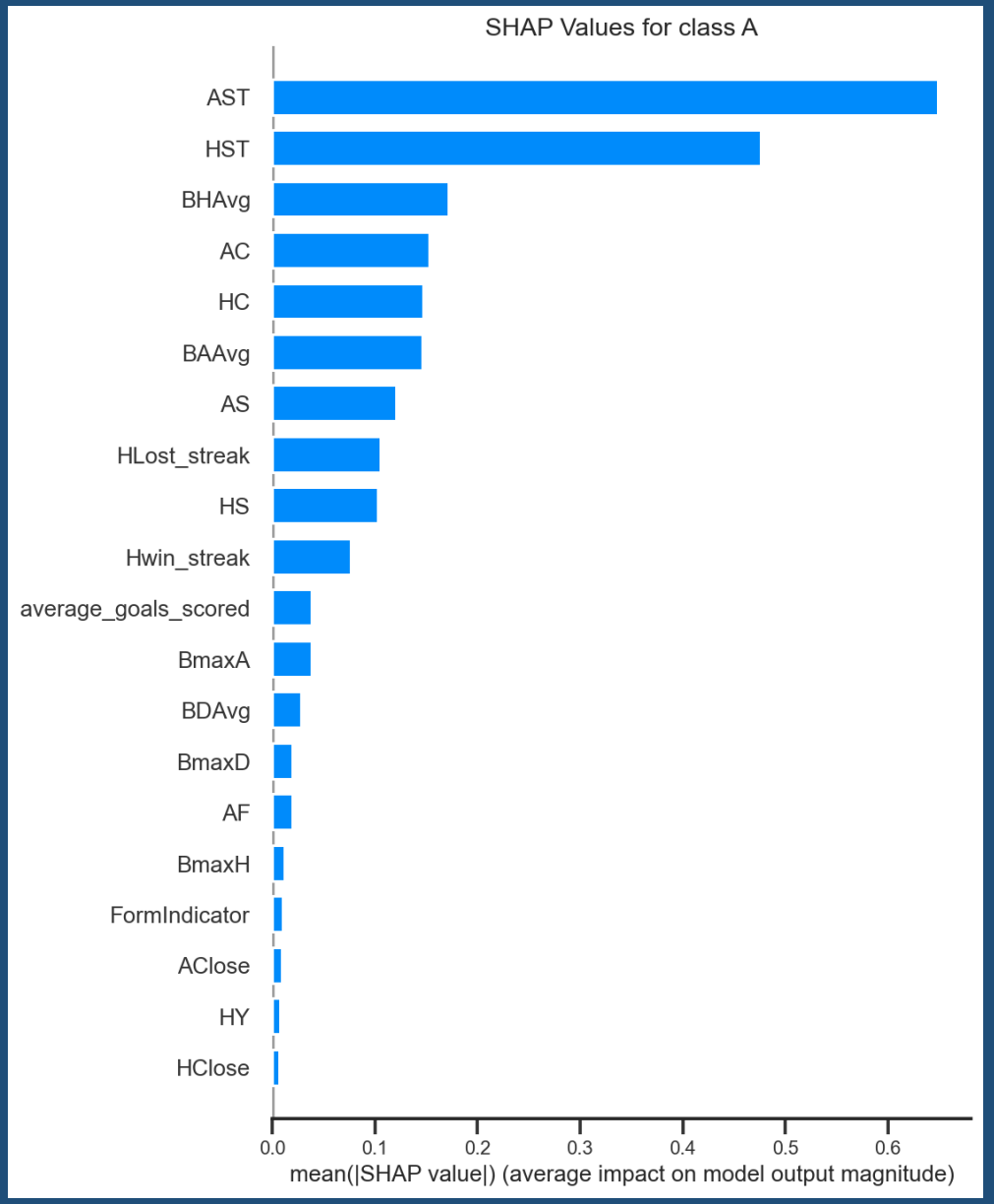
Feature Contribution for the Home Wins



# Model Evaluation cont…

(SHAP Analysis cont…)

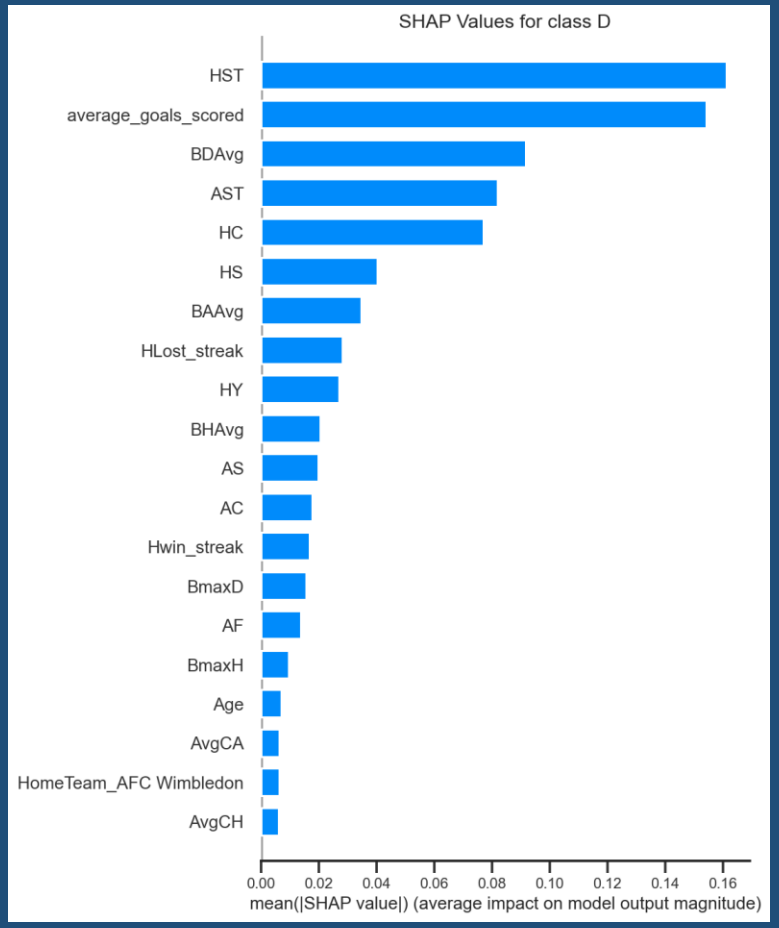
- Feature Contribution for the Away Win



# Model Evaluation cont…

(SHAP Analysis cont…)

- Feature Contribution for the Draws



# Conclusion






- Literature from Over 50 Journals of the last 10 yrs.
- Extracted EPL data with 45952 observations.
- Investigated clustering and ELO rating. Feature engineering and selection were done, and SMOTE resampling worked well.
- The selected five structurally diverse models with ensemble and DL models were trained, and validated and the best models were tested where RFC and VC models lead.
- Objectives were evaluated based on Accuracy, Precision, Recall, F1-score, CV scores, ROC-AUC, and True vs. Predicted Analysis to reveal the most robust model.
- Accordingly, the RFC showed 66% accuracy on testing data, with more robust other indices.
- SHAP analysis was done to figure out the feature contribution towards each class of match outcome.
- Recommended RFC for model applications & and future works.